Aesthetic Computer: Open Development

A Platform Submitted for Dialogue

Jeffrey Scudder / @jeffrey

Score for Social Software — Cycle 2 · DESMA 596/199 · March 2026

aesthetic.computer · github · nopaint.art

THE SCORE

I'm submitting Aesthetic Computer (AC) — the open-source creative computing platform I've been building since 2021 — as a social software project in active development. I'm not proposing to build a new thing for this cycle. I'm bringing the thing I'm already building into the room for dialogue, feedback, and critical exchange as I continue to develop it alongside its community.

AC is a mobile-first runtime and social network where users write, publish, and share small interactive programs called *pieces*. It has real-time chat, @handles with per-character color customization, ephemeral status updates (moods mirrored to Bluesky via ATProto), a pixel painting system, a built-in Lisp dialect (KidLisp), multiplayer WebSocket sessions, and user profiles that track creative output. The codebase is open source with 4+ years of continuous development.

The score is simple: I develop AC in the open over 10 weeks, sharing what I'm working on, what decisions I'm facing, and what the community is doing on the platform. The cohort engages as users and as critics — trying the tools, reading the design choices, and giving me feedback I can't get from inside the project.

WHAT I'M LOOKING FOR

I've been deep inside AC's architecture for years. I know how the systems work. What I lack is outside perspective on the social design — the questions I can't answer alone:

- What feels inviting and what feels opaque when you first encounter the platform?
- Which social features sustain participation vs. which are technically impressive but socially inert?
- How does the "instrument" metaphor land for people who aren't already invested in it?
- What would you want to do on AC that you currently can't?
- Where does the design accidentally exclude the people it claims to welcome?

These questions require sustained dialogue, not a single usability test. A cohort of practitioners thinking about social software from different angles is the right context for this kind of exchange.

WHAT THE COHORT GETS

- **A live codebase** — open source, documented, actively changing week to week
- **The dev process** — I work with Claude Code, Cursor, and VS Code, and maintain a living `SCORE.md`; the process is as legible as the product
- **Platform accounts** — everyone gets an @handle, can paint, chat, set moods, write KidLisp, publish pieces
- **Real community data** — 2,800 handles, 18k chat messages, 4,400 paintings, 16k KidLisp programs; the social patterns are there to observe

WHY THIS CYCLE

Casey's framing of "scores for social software" maps directly onto how I already think about AC.

The platform's `SCORE.md` literally uses the metaphor of a musical score to organize the project. The interface is designed to work like an instrument — users discover memorizable paths, build literacy through play, and eventually improvise. But I've been composing alone. This cycle is a chance to compose in conversation.

I'm at a specific inflection point. The technical infrastructure is mature. The question now is about social design: how do the features I've built actually shape the way people relate to each other and to creative computing? That's best answered through dialogue with people thinking critically about social software, not through more engineering.

PRACTICE

I'm an artist, educator, and software developer. Before AC, I created No Paint (2020), a pixel art tool whose community of non-technical users taught me how people learn computing through social participation in software they love. AC extends that into a full platform: anyone can write, publish, and share interactive programs at a URL.

I teach creative computing and have used AC as infrastructure in courses and workshops. My interest in social software comes from watching people learn computation through social participation — first in No Paint, now in AC. I'm bringing this not as a finished project but as an ongoing practice I want to develop through critical exchange.

APPENDIX A: THE NETWORK

Live data from AC's MongoDB cluster — March 2, 2026

2,798 @HANDLES	18,016 CHAT MESSAGES	4,392 PAINTINGS
2,900 MOODS	16,174 KIDLISP PROGRAMS	265 PUBLISHED PIECES
333 CLOCKS	102 TAPES	93,122 BOOT EVENTS

Who makes things: 1,067 have painted · 997 have posted moods · 59 have written KidLisp · 19 have published pieces

THE INSTRUMENT LOOP

prompt → type a piece name → enter → play the piece → esc → back to prompt

USER DATA FLOW

@handle → paint / chat / mood / kid / publish → MongoDB → profile → public URL

moods → Bluesky via ATProto · paintings → DO Spaces CDN

SAMPLE DOCUMENTS (LIVE)

```
mood  { mood: "studying astronomy", when: "2026-03-02T12:32:49Z", atproto: { rkey: "3mg3b6jcj4k2x" } }
```

```
painting  { code: "gfl", slug: "2026.03.02.14.14.39", when: "2026-03-02T13:14:48Z" }
```

```
kidlisp  { code: "27z", source: "fade:red-blue-black-blue-red\nscroll (* 100 amp)", hits: 1 }
```

```
chat  { text: "y'all gonna piss me off", when: "2026-03-02T05:26:25Z", font: "font_1" }
```

```
piece  { code: "zod", slug: "zod", name: "3d-cube", extension: ".mjs", hits: 1 }
```

APPENDIX B: TECHNICAL STACK

Open source — github.com/whistlegraph/aesthetic-computer

FRONTEND RUNTIME

- ES Modules (.mjs) — no build step for pieces
- Canvas 2D + WebGL2 rendering
- WebSocket hot-reload module loader
- Service worker for offline caching
- 351 built-in pieces (333 JS + 18 KidLisp)

SYSTEM SERVER (NETLIFY)

- ~85 serverless API endpoints
- Edge functions for media + routing
- Auth0 + Firebase authentication
- Stripe + PayPal payments
- Auto-deploys on push to main

SESSION SERVER

- Fastify + Geckos.io (WebSocket)
- Jamsocket ephemeral containers
- Real-time chat, multiplayer, rooms
- Redis state synchronization
- DigitalOcean droplet + pm2

OVEN SERVICE

- Express.js + FFmpeg
- Tape recordings → MP4 video
- Screenshot & OG image generation
- Caddy reverse proxy (auto HTTPS)
- DigitalOcean droplet

WORKERS (CLOUDFLARE)

- Feed server (Hono + TypeScript)
- Grab worker (Browser Rendering API)
- KV storage for playlists & channels
- Durable Objects for state

DATA

- MongoDB Atlas — all user content
- Redis — session cache
- DigitalOcean Spaces (S3 CDN) — media
- Cloudflare KV — feed data
- ATProto PDS — Bluesky bridge

KIDLISP — BUILT-IN LANGUAGE

118 built-in functions across 12 categories: math, color, shape, transform, animation, text, noise, input, logic, list, meta, system. Minimal Lisp dialect for generative art. No external libraries. Programs stored in MongoDB, shareable by 3-character code. Evaluator: **lib/kidlisp.mjs** (~2,400 lines).

DEVELOPMENT TOOLS

Claude Code (Opus) · Cursor · VS Code · Node.js · esbuild · Puppeteer · Jasmine · Vitest · Lighthouse · Netlify CLI · Wrangler · Docker · pm2 · Caddy · systemd

SOURCE CODE

Repository: github.com/whistlegraph/aesthetic-computer · Open source, continuously developed since 2021
Core files: boot.mjs (entry) → bios.mjs (runtime) → disk.mjs (API, ~572KB) → disks/*.mjs (pieces)
Languages: JavaScript (ES Modules), TypeScript (workers), KidLisp (generative art), HTML/CSS
Project score: SCORE.md — a living musical score metaphor that organizes all development work
Maintenance: AestheticAnts (AA) — colony of AI agents for automated codebase care

---

APPENDIX C: SITEMAP

22 domains · ~355 disk routes · ~85 API endpoints · ~120 redirects

aesthetic.computer primary platform	kidlisp.com lisp editor	notepat.com music sequencer
sotce.net art portal	botce.ac bot interface	wipppps.world whistlegraph
news.* newsletter	give.* donations	shop.* physical goods
help.*	feed.*	grab.*

AI assistant | DP1 feed | screenshots

CREATIVE TOOLS / DRAWING (44)

/paint /nopaint /line /pline /shape /oval /rect /fill /spray /smear /marker /crayon /sparkle-brush /multipen /doodle /wand /brush /crop /stamp /blur /snap /paste /make /painting /handprint /nail /colors /selfie /camera /cards

MUSIC / AUDIO (24)

/notepat /beat /tone /chord /melody /song /sing /bleep /say /whistle /metronome /microphone /uke /pedal /amp /sfx /r8dio /squaresong /rattle /visualizer /tracker /3x3

GAMES / INTERACTIVE (25)

/game /1v1 /brick-breaker /flap /fly /hop /scawy-snake /run&gun /gostop /staka /pond /balls /bubble /rain /sno /starfield /metaballs /ant /field /toss

KIDLISP / PROGRAMMING (15)

/kidlisp /keep /kept /learn /lang /decode /encode /prompt /chat /docgen /docs/*

SOCIAL / COMMUNITY (8)

/mood /moods /share /sign /signature /mail /list /insta

PEOPLE / CHARACTERS (16)

/mom /dad /brother /sister /husband /wife /boyfriend /girlfriend /kid /angel /robo /sage /tobby /valbear /gargoyle /dolls

WHISTLEGRAPH (8)

/whistlegraph /wg /wgr /m2w2 /wipppps /neo-wipppps /merry-fade

MEDIA / TV (7)

/tv /tapes /screen /screenshots /replay /video

UCLA COURSE (13)

/ucla-1 /ucla-2 /ucla-3 /ucla-4 /ucla-5 /ucla-6 /ucla-7
+ keyboard, box, turtle, balls, dial, jump variants

SINGLE-LETTER & NUMERIC (47)

/$ /0-9 /a-z

DYNAMIC PATTERNS

/@:handle → user profile · /@:handle/:piece → user piece · /preview/* → OG images · /session/* → multiplayer rooms